POLITECNICO MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE E DELL'INFORMAZIONE

MRN O-RAN Project: Milestone 2

BER-based Downlink MCS Control xApp

Tesi di Laurea Magistrale in Telecommunication Engineering
Ingegneria delle telecomunicazioni

Author: **Hong Chen**

GitHub repository

Student ID: 11024836
Advisor: Prof. Antonio Capone
Co-advisors: Ph.D. Alberto Ceresoli

Academic Year: 2025–2026

1 | MRN O-RAN Project – Milestone 2

1.1. Protocol Design and Message Flow

This work implements Project 2: a near-RT RIC xApp monitors the downlink BER of each connected UE and sends a control request to the gNB emulator to set the target downlink MCS. The implementation extends the protobuf-based E2SM used by the provided xApp and gNB emulator. The closed loop is shown in Fig. 1.1: the gNB reports per-UE BER and MCS state, the xApp applies a threshold-based decision, and the resulting target MCS is sent back as an E2 control message.

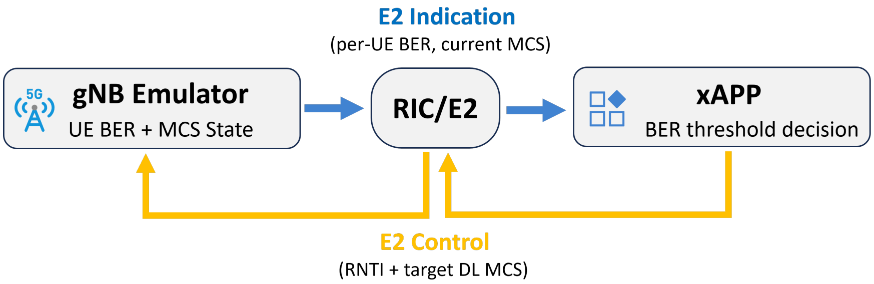

Figure 1.1: Closed-loop BER-based downlink MCS control through the RIC/E2 interface.

The existing `UE_LIST` parameter is used as the container for all per-UE monitoring and control values. This keeps each measurement and control action associated with the UE RNTI.

Field	Direction	Meaning
`rnti`	both	UE identifier used to match measurements and controls.
`dl_ber`	gNB $\rightarrow$ xApp	Downlink BER monitored by the xApp.
`dl_mcs`	gNB $\rightarrow$ xApp	Current downlink MCS stored by the emulator.
`target_dl_mcs`	xApp $\rightarrow$ gNB	Target downlink MCS requested by the xApp.

The protobuf extension keeps the UE-level design compact. The `rnti` identifies the UE, `dl_ber` is the monitored value, `dl_mcs` reports the current emulator state, and `target_dl_mcs` carries the MCS control decision back to the gNB.

```
message ue_info_m {
  required int32 rnti = 1;
  optional float dl_ber = 7;
  optional int32 dl_mcs = 8;
  optional int32 target_dl_mcs = 9;
}
```

1.2. Implementation

On the gNB emulator side, each UE is stored in memory with its RNTI, current BER, current downlink MCS, and latest target downlink MCS. The emulator reports four UEs and alternates BER between 0.05 and 0.20, so both sides of the 0.10 threshold are tested. When the xApp requests `UE_LIST`, the gNB returns a `ue_info_m` entry for each UE. When a control request is received, the gNB extracts `rnti` and `target_dl_mcs`, then updates the matching UE state.

The xApp is configured with a 0.10 BER threshold, target low MCS 4, normal MCS 20, and a 500 ms monitoring interval. In this run, UEs satisfying the BER condition are assigned MCS 4; the others are restored to MCS 20.

```
python3 myxapp.py \
  --ber-threshold 0.10 \
  --target-low-mcs 4 \
  --normal-mcs 20 \
  --interval 0.5
```

Every 500 ms, the xApp checks received RIC indications. For each UE, it reads `dl_ber`, compares it with the threshold, and selects either the low or normal MCS. If the selected value differs from the current target reported by the gNB, the xApp sends a `CONTROL` message carrying the UE RNTI and the new `target_dl_mcs`.

1.3. Validation Evidence

The experiment was run with `dev-compose.yaml`. The gNB E2SM server, E2AP agent, and monitoring/control xApp were started in separate terminals. Fig. 1.2 shows the two key outputs: the xApp monitors BER and sends a control request; the gNB receives the control request and applies the requested target MCS values to multiple UEs.

```
1 messages received while waiting, printing:
RIC Indication received from gNB b'gnb_734_733_b5c67780', decoding E2SM payload
UE 17921: dl_ber=0.0500, dl_mcs=20, target_dl_mcs=20, decision=FORCE_LOW_MCS
UE 17922: dl_ber=0.2000, dl_mcs=4, target_dl_mcs=4, decision=NORMAL_OPERATION
UE 17923: dl_ber=0.0500, dl_mcs=20, target_dl_mcs=20, decision=FORCE_LOW_MCS
UE 17924: dl_ber=0.2000, dl_mcs=4, target_dl_mcs=4, decision=NORMAL_OPERATION
control request encoded 704 bytes
Sent control request with 4 UE MCS update(s)
```

(a) xApp: BER monitoring, decision, and control request.

```
Received 8 bytes
ran message id 2
Indication request message received
Indication request for 2 parameters:
        Parameter id 1 requested (a.k.a gnb_id)
        Parameter id 3 requested (a.k.a ue_list)
Sending indication response
Sent 145 bytes, buflen was 145

------------------------------
Received 44 bytes
ran message id 4
Control message received
Applying target parameter ue_list
Applied DL MCS target 20 to RNTI 17921
Applied DL MCS target 4 to RNTI 17922
Applied DL MCS target 20 to RNTI 17923
Applied DL MCS target 4 to RNTI 17924
```

(b) gNB: control received and MCS applied.

Figure 1.2: xApp and gNB terminal evidence for the Project 2 closed loop.

The logs demonstrate the required pass condition: the gNB reports BER values higher and lower than the threshold, the xApp reacts by sending a control request, and the gNB stores the received target MCS values for multiple UEs. The implementation is intentionally limited to the emulator, where the MCS state is stored in memory.